

# MP3: Conway's Game of Life on WS2812B

---

Eddy Pan 30 October 2025

In this Mini-Project, I play three of Conway's Game of Life simultaneously on a WS2812B 8x8 RGB LED Matrix--one for each color channel.

## Adaptable Sample Code

Much of this project was adapted from the `led_matrix` spiral example. I re-used the `ws2812b`, `controller` and parts of the `top` module to implement the game of life.

### Reused `ws2812b` Module

This module uses a two-state state machine to handle the timing protocol required by WS2812B LEDs. It converts serial RGB data into the specific format the WS2812B matrix expects.

### Reused `controller` Module

This module orchestrates the display timing and frame management for the LED matrix. Also as a two-state state machine, it alternates between transmitting pixel data to the WS2812B LEDs, and idling between frames. An important change that I made was modifying the `IDLE_CYCLES` calculation to match my desired frame rate of 1 frame a second. This required upping `IDLE_CYCLES` from

$(12,000,000 / 32) - 64 \times (360 + 2) = \mathbf{351,832}$  cycles for 32 frames / second, to

$(12,000,000 / 1) - 64 \times (360 + 2) = \mathbf{11,976,832}$  cycles for 1 frame / second.

## Implementation

Throughout this section, I'll be mainly referring to the `green_*` Game of Life implementation. Since the `red_*` and `blue_*` Game of Life modules are implemented the same way as `green_*`, I dive into the implementation of just the `green_*` module to reduce redundancy.

### `game_of_life` module

This module takes in an initial state (`init_state`), clock (`clk`), and `start` flag to calculate what its current state (`curr_state`) should be after an iteration of the Game of Life's rules. In addition to `curr_state`, it also outputs flags for `done` and `updating`.

I first instantiate three logics-- a 64-bit `next_state` temporary board, an `index` counter, and a register to count `alive_neighbors` of any given cell. I have stored all of my game boards in 64-bit registers as they are very compact representations, and despite being 1-dimensional, I'm still able to efficiently compute any cell's neighbors.

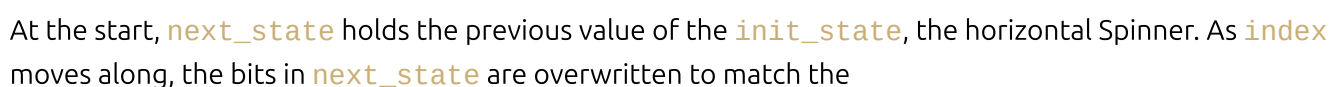
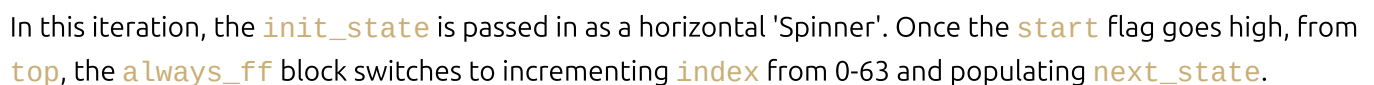
Next is an `initial` block that ensures the `done` and `updating` flags are set to 0.

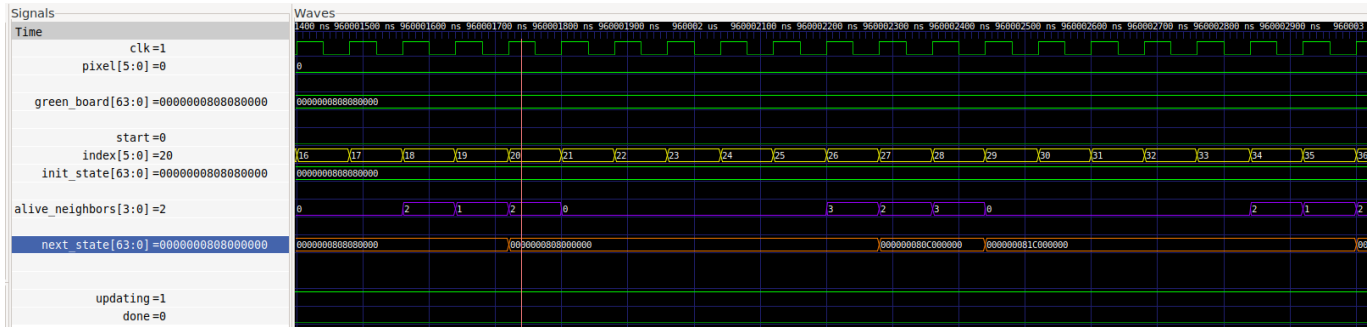
The main logic occurs in these next two `always` blocks. The first is an `always_comb` which calls the `count_alive_neighbors` function and stores the result in the `alive_neighbors` register. The

## Conway's Game of Life Rules

- **Birth:** A dead cell with exactly 3 live neighbors becomes alive
- **Survival:** A live cell with 2 or 3 live neighbors stays alive
- **Death:** All other cells die or stay dead

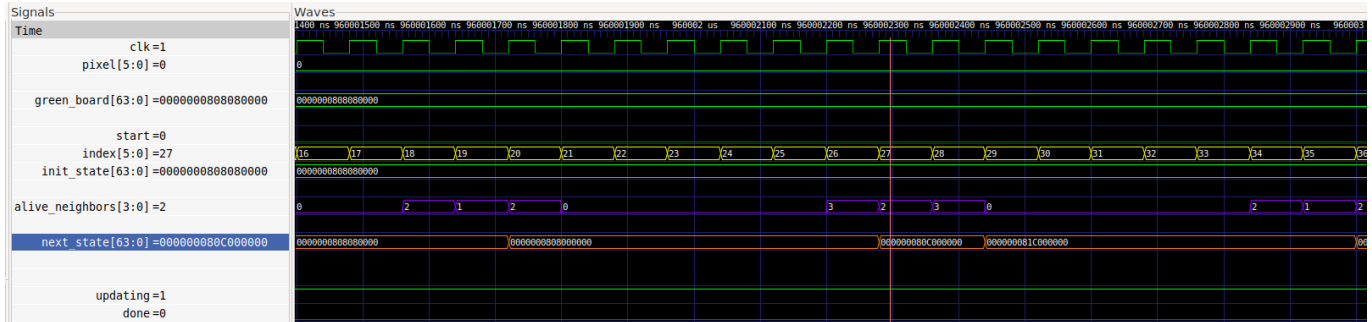
Here's what the simulation looks like for one iteration:





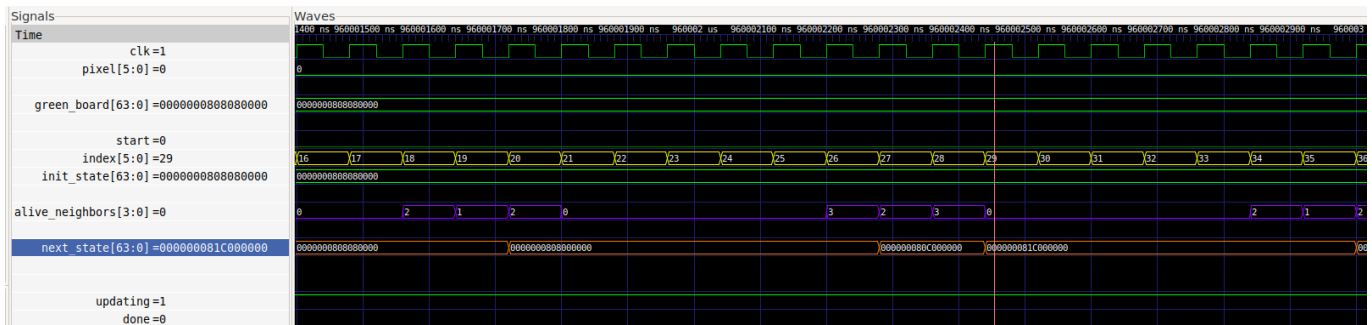
At about index 20, a hexadecimal **8** (4b'0100) is replaced by a **0** (4b'0000).

At index 26, **alive\_neighbors** reaches a **3**, meeting a condition to be alive,

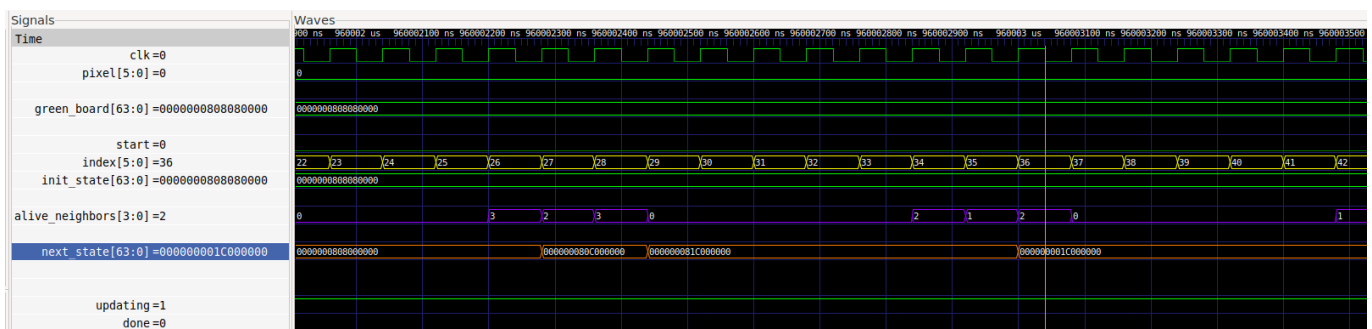


the hexadecimal changes from **8** (4b'1000) to **C** (4b'1100). At index 27, since **alive\_neighbors == 2** and **init\_state[27] == 1**, the cell stays alive, and stays at **C** (4b'1100) instead of dropping to **4** (4b'0100).

Finally, at index 27, **alive\_neighbors == 3**,

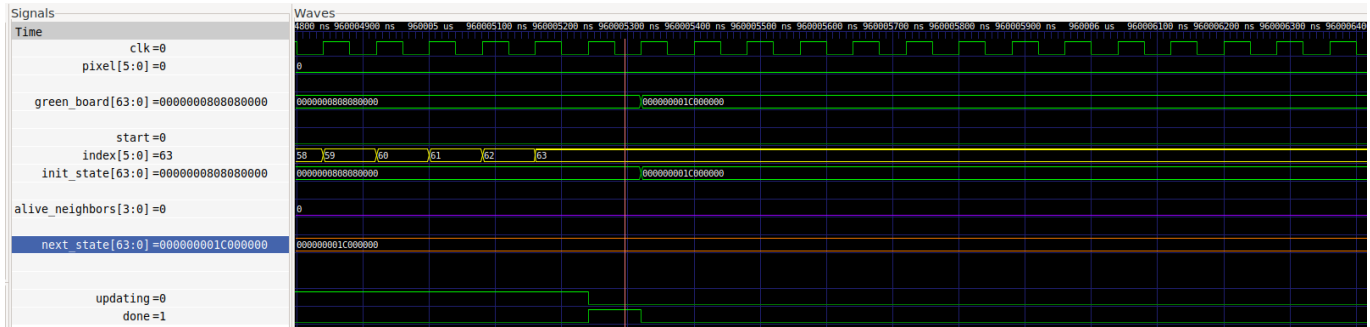


the next hex **0** (4b'0000) turns into a **1** (4b'0001).



Finally, by the 36th index, the last hex from the previous state **8** (4b'0100), is switched off into a **0** (4b'0000).

At the end of the 63rd index, the updating flag turns off, and the done flag goes up. **green\_board** is then updated to **next\_state**.



A potential next-step would be to set the state of the 64-bit register to all 0s and then populate it at each frame so that the data from the last state's bits aren't pre-populated--rendering the simulation results a bit easier to read and debug.

## top module

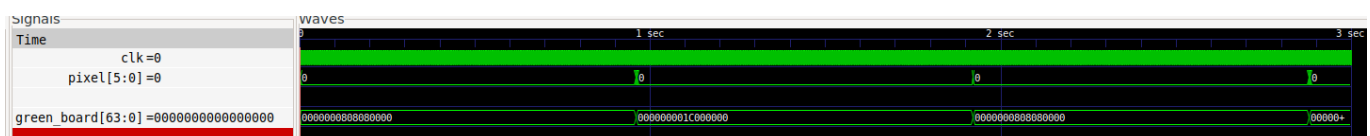
I organized the `top` module similarly to the `led_matrix` example. Since I keep track of each of my game states in a 64-bit register, where each bit represents if the cell at that index is alive or dead (1 or 0), I removed the `memory` modules. To initialize my game state from a file, I used an initial block with `$readmemh` to read in the initial 64-bits of each text file into an 1-element unpacked temporary array, `green_init_board`. In the `always_ff` block right after it that executes on the positive clock edge, I check a `board_initialized` flag to determine whether I initialize my `green_board` with the 64-bits from the `green.txt` file, or update my `green_board` to `green_next` computed from my `game_of_life` module.

With my `game_board` update logic handled, I instantiate the three `game_of_life` modules to calculate the next game state for the `red_board`, `blue_board`, and `green_board`. These `game_of_life` modules use shared `game_start`, `game_done`, and `game_updating` states to keep the code a bit more DRY and to keep my three game-of-life modules synced up.

After instantiating the `ws2812b` and `controller` modules just as they were in the `led_matrix` example, I have an `always_ff` block that triggers on the positive edge of the clock to load the shift register from the `green`, `red`, and `blue` board states when the `load_sreg` signal is high. For each 8-bit segment (G,R,B) the shift register is loaded with either `8'h0F` or `8'h00` (on or off) depending on whether the cell at that pixel is alive.

## Testing and Validation

Using the GTKWave test, I ran my test bench for 3 seconds to capture 3 iterations of the Game of Life. Here are some results:



I tested my Game of Life implementation by initializing known "stable" patterns in my board initialization text files in `initial_led_state/`, and verifying that the next state matched the expected next iteration for that stable state.

For example, in the image above, I am modeling the 'Spinner' pattern as a test case for validating my Game of Life code. In the image above, my pattern starts as:

**Initial State:** 0x00\_00\_00\_00\_1C\_00\_00\_00

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |                            |
|---|---|---|---|---|---|---|---|---|----------------------------|
| 0 | . | . | . | . | . | . | . | . |                            |
| 1 | . | . | . | . | . | . | . | . |                            |
| 2 | . | . | ■ | ■ | ■ | . | . | . | ← Row 2: bits 16-23 = 0x1C |
| 3 | . | . | . | . | . | . | . | . |                            |
| 4 | . | . | . | . | . | . | . | . |                            |
| 5 | . | . | . | . | . | . | . | . |                            |
| 6 | . | . | . | . | . | . | . | . |                            |
| 7 | . | . | . | . | . | . | . | . |                            |

and transitions into its

**Next State:** 0x00\_00\_00\_00\_08\_08\_08\_00

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |                        |
|---|---|---|---|---|---|---|---|---|------------------------|
| 0 | . | . | . | . | . | . | . | . |                        |
| 1 | . | . | . | ■ | . | . | . | . | ← Row 1: bit 19 = 0x08 |
| 2 | . | . | . | ■ | . | . | . | . | ← Row 2: bit 27 = 0x08 |
| 3 | . | . | . | ■ | . | . | . | . | ← Row 3: bit 35 = 0x08 |
| 4 | . | . | . | . | . | . | . | . |                        |
| 5 | . | . | . | . | . | . | . | . |                        |
| 6 | . | . | . | . | . | . | . | . |                        |
| 7 | . | . | . | . | . | . | . | . |                        |

In simulation, the Spinner repeats over and over again between horizontal and vertical, which matches its expected behavior and thus validates this Game of Life implementation for this test case.

As further validation, in addition to the Spinner, I also tested the 'Glider' pattern, which is expected to move across the board and not die out. I set it up on the red channel, and in the video, I demonstrate its behavior of traveling through the cells on its diagonal.

## Results

View the video labeled [CompArch\\_MP3\\_demo.mov](#) to see the working representation of this Game of Life implementation on the WS2812B.